Kmer graph practical cheatsheet

Replace uppercase filenames with your input filenames. `OUTPUT` names a new output directory unless a tool uses it as a filename prefix. Each command below is one shell line. Choose new output names for separate runs.

1 Course Docker image

The course image contains the tools, but no course data. In your computer's terminal, open the `Scalable_Genomics_and_Pangenomics_2026` repository folder. It must contain `course_data_2026`.
Run this command from that folder. It works on Linux, macOS and Windows PowerShell.

```
docker run --rm -it --platform linux/amd64 -p 4242:4242 -v "${PWD}:/course" -w /course npmalfoy/scalable:2026
```

`${PWD}` is your current repository folder. Keep the braces, especially in PowerShell. The folder is shared with the container at `/course`, where the terminal starts.
In the terminal opened by Docker, enter the practical data folder:

```
cd course_data_2026/Kmers_in_a_Graph
```

Run the tool commands there. **Save outputs inside `/course`**, for example in the practical data folder. These files remain on your computer after exit. The container is removed when you exit. Each `docker run` starts a new container and shares the same repository folder again.
`--platform linux/amd64` selects the image architecture. Port 4242 lets your browser connect to Vizitig.

2 Assemblers

2.1 SPAdes

Assemble paired end Illumina reads with the default kmer sizes

```
spades.py --only-assembler -1 READS_R1.FQ.gz -2 READS_R2.FQ.gz -t 4 -m 8 -o OUTPUT
```

For these clean synthetic reads, `--only-assembler` skips read error correction. Both mate files must contain matching read pairs in the same order.

Main outputs, inside OUTPUT:

- `contigs.fasta`: assembled sequences. Use this first for contig statistics and evaluation.
- `scaffolds.fasta`: contigs joined using linking evidence. Unknown stretches appear as `N`.
- `assembly_graph_with_scaffolds.gfa`: the assembly graph and scaffold paths.

Use one kmer size for a controlled comparison

```
spades.py --only-assembler -1 READS_R1.FQ.gz -2 READS_R2.FQ.gz -k 31 -t 4 -m 8 -o OUTPUT
```

Ignore pairing while keeping exactly the same input bases

```
spades.py --only-assembler --s1 READS_R1.FQ.gz --s2 READS_R2.FQ.gz -k 31 -t 4 -m 8 -o OUTPUT
```

Use a different `OUTPUT` for each run. Compare contig and scaffold statistics separately.

Increase the kmer size with paired end reads

```
spades.py --only-assembler -1 READS_R1.FQ.gz -2 READS_R2.FQ.gz -k 127 -t 4 -m 8 -o OUTPUT
```

Repeat the larger kmer experiment without pairing

```
spades.py --only-assembler --s1 READS_R1.FQ.gz --s2 READS_R2.FQ.gz -k 127 -t 4 -m 8 -o OUTPUT
```

Combine several kmer sizes

```
spades.py --only-assembler -1 READS_R1.FQ.gz -2 READS_R2.FQ.gz -k 21,33,55,77,99,127 -t 4 -m 8 -o OUTPUT
```

2.2 MEGAHIT

Assemble paired end Illumina reads with the default settings

```
megahit -1 READS_R1.FQ.gz -2 READS_R2.FQ.gz -t 4 -m 8589934592 -o OUTPUT
```

Main output: `final.contigs.fa` contains the assembled contigs. Use it for evaluation and graph construction. Here `-m 8589934592` sets an 8 GiB memory budget in bytes.

Choose the kmer sizes

```
megahit -1 READS_R1.FQ.gz -2 READS_R2.FQ.gz --k-list 21,41,61,81,101,121 -t 4 -m 8589934592 -o OUTPUT
```

Keep only contigs of at least 500 bases

```
megahit -1 READS_R1.FQ.gz -2 READS_R2.FQ.gz --min-contig-len 500 -t 4 -m 8589934592 -o OUTPUT
```

2.3 Flye

Assemble the Oxford Nanopore reads

```
flye --nano-hq NANOPORE.FQ.gz --genome-size 500k --threads 4 --out-dir OUTPUT
```

`--nano-hq` suits the practical's synthetic Nanopore reads with approximately 1% errors.

Main outputs, inside OUTPUT:

- `assembly.fasta`: final assembled sequences. Use this for evaluation.
- `assembly_info.txt`: contig length, coverage and circularity information.
- `assembly_graph.gfa`: the final assembly graph.

Assemble PacBio high fidelity reads instead

```
flye --pacbio-hifi PACBIO.FQ.gz --genome-size 500k --threads 4 --out-dir OUTPUT
```

2.4 hifiasm

Assemble the PacBio high fidelity reads

```
hifiasm -o OUTPUT -t 4 -l0 -f0 PACBIO.FQ.gz
```

Here `OUTPUT` is a filename prefix. For these small haploid genomes, `-l0` disables duplicate purging and `-f0` disables the Bloom filter.

Main outputs:

- `OUTPUT.bp.p_ctg.gfa`: primary contig sequences, stored as graph segments. Start with this file.
- `OUTPUT.bp.r_utg.gfa`: the raw unitig graph, useful for investigating unresolved branches.

Extract primary contig sequences into a FASTA file

```
awk '$1=="S" && $3!="*" {print ">"$2; print $3}' ASSEMBLY.GFA > ASSEMBLY.FA
```

Set `ASSEMBLY.GFA` to the primary contig output. `ASSEMBLY.FA` contains its sequences and can be evaluated with QUAST or MerquryFK. This extraction does not retain graph links.

Combine PacBio high fidelity reads with ultralong Oxford Nanopore reads

```
hifiasm -o OUTPUT -t 4 -l0 -f0 --ul NANOPORE.FQ.gz PACBIO.FQ.gz
```

`--ul` supplies the Nanopore reads. The final input supplies the PacBio high fidelity reads. Use reads from the same sample and a new output prefix for the hybrid run. Compare its primary contigs with the PacBio only assembly.

2.5 Verkko

Combine PacBio high fidelity reads and Oxford Nanopore reads

```
verkko -d OUTPUT --hifi PACBIO.FQ.gz --nano NANOPORE.FQ.gz --no-correction --no-consensus-bam --haploid --local-cpus 4
```

`--local-cpus 4` uses four processor threads. Add `--local-memory 8` to set an 8 GB local memory budget.

The practical uses `--no-correction` for its already accurate synthetic high fidelity reads. `--no-consensus-bam` avoids a known failure in the small genome recipe. `--haploid` matches the simulated genomes.

Main outputs, inside OUTPUT:

- `assembly.fasta`: final assembled sequences. Use this first for quality evaluation.
- `assembly.homopolymer-compressed.gfa`: an assembly graph with homopolymer runs compressed. It is not the final full length sequence.

Assemble PacBio high fidelity reads without Nanopore reads

```
verkko -d OUTPUT --hifi PACBIO.FQ.gz --no-nano --no-correction --no-consensus-bam --haploid --local-cpus 4 --local-memory 8
```

`--no-nano` selects the HiFi only workflow. The practical's Nanopore like reads alone cannot replace the accurate input required through `--hifi`.

3 Read and assembly statistics

3.1 SeqKit

Write the supplied reference as an uncompressed FASTA

```
seqkit seq REFERENCE.FA.gz > REFERENCE.FA
```

Use `REFERENCE.FA` in the QUAST and graph comparison commands below.

Get basic read or assembly statistics

```
seqkit stats READS.FQ.gz
seqkit stats ASSEMBLY.FA
```

The terminal output lists sequence count, total bases, and minimum, mean and maximum sequence lengths.

Include N50 and additional length and quality statistics

```
seqkit stats -a READS_R1.FQ.gz READS_R2.FQ.gz PACBIO.FQ.gz NANOPORE.FQ.gz
seqkit stats -a ASSEMBLY.FA
```

N50 is the sequence length at which sequences of that length or longer account for at least half of the total sequence length. For paired end depth, count the bases in both mate files.

Compare assembly sequences of at least 500 bases

```
seqkit seq -m 500 ASSEMBLY.FA | seqkit stats -a
```

This filters complete FASTA records by length. It does not split scaffolds at unknown bases, so use contig FASTA files for a contig comparison.

Write a statistics table to a file

```
seqkit stats -a -T ASSEMBLY_1.FA ASSEMBLY_2.FA > STATISTICS.tsv
```

`STATISTICS.tsv` is a tab separated table that can be opened in a spreadsheet. Compare sequence count, total bases, N50 and longest sequence alongside correctness metrics.

4 Assembly evaluation

4.1 QUAST

Compare one assembly with the related reference

```
quast.py ASSEMBLY.FA -r REFERENCE.FA --min-contig 500 --threads 4 -o OUTPUT
```

`REFERENCE.FA` is the supplied related reference. Differences can reflect genomic variation as well as assembly errors. Use read based evaluation and graph exploration to investigate them.

Main outputs, inside OUTPUT:

- `report.html`: the main interactive report. Open this file in a web browser first.
- `icarus.html`: contig alignment viewer. Inspect missing regions, repeated sequence and suspicious joins.
- `report.tsv`: tab separated metrics for comparing metrics or opening in a spreadsheet.
- `report.pdf`: printable summary and plots.

Compare several assemblies with the related reference

```
quast.py ASSEMBLY_1.FA ASSEMBLY_2.FA ASSEMBLY_3.FA -r REFERENCE.FA --min-contig 500 --threads 4 -o OUTPUT
```

Metrics to inspect:

- **Genome fraction:** how much of the reference is covered by assembly alignments.
- **Duplication ratio:** aligned assembly bases relative to covered reference bases. Excess sequence can indicate redundancy.
- **Misassemblies:** inconsistencies in aligned contig structure. Inspect the alignment evidence.
- **NGA50:** aligned block contiguity, using reference length and splitting blocks at extensive misassemblies.
- **Mismatches and indels per 100 kb:** nucleotide disagreement with the reference.

The reference is circular but stored as a linear FASTA. Keep QUAST's default circular origin handling. Do not add `--fragmented` for this practical.

4.2 FastK and MerquryFK

Count kmers from both Illumina mate files

```
FastK -k21 -t1 -M2 -T4 -NREADS READS_R1.FQ.gz READS_R2.FQ.gz
```

`-k21` counts 21mers. `-t1` retains every kmer occurring at least once, as required by MerquryFK. `-T4` uses four threads and `-M2` sets a 2 GB sorting memory budget. `-NREADS` sets the output prefix. Pairing information is not used.

`READS.ktab` is the kmer table. `READS.hist` is a binary histogram. Keep these and the hidden `.READS.ktab.*` files together. Use reads from the same sample as the assembly.

Evaluate one assembly without a reference genome

```
MerquryFK -T4 READS ASSEMBLY.FA EVALUATION
```

`READS` identifies the FastK table and histogram. `EVALUATION` is an output filename prefix. Use a new prefix for each run. No `MERQURY` environment variable is needed.

Main outputs:

- `EVALUATION.qv`: estimated consensus quality and unsupported kmer counts.
- `EVALUATION.completeness.stats`: recovery of reliable read kmers. This is not the percentage of genome bases recovered.
- `EVALUATION.ASSEMBLY.spectra-cn.ln.png`: copy number spectrum, with `.fl.png` and `.st.png` for filled and stacked views.
- `EVALUATION.spectra-asm.ln.png`: read kmers present or absent in the assembly.
- `EVALUATION.ASSEMBLY.qv`: quality for each contig. `EVALUATION.ASSEMBLY_only.bed`: positions with unsupported assembly kmers. `ASSEMBLY` is the input filename without its extension.

Export the read kmer multiplicity distribution

```
Histex -A -h1:500 READS > READS.hist.tsv
```

`READS.hist.tsv` lists multiplicity and the number of distinct kmers for counts 1 to 500. `-A` exports text. Low count kmers often include sequencing errors. Repeated sequence can form higher multiplicity peaks.

Keep plotting data and show assembly only kmers

```
MerquryFK -T4 -k -l -z READS ASSEMBLY.FA EVALUATION_DETAIL
CNplot -l -z -X250 EVALUATION_DETAIL.ASSEMBLY.spectra-cn.cni
```

`-k` keeps plot data, `-l` selects a line plot and `-z` shows assembly only kmers at zero. The second command redraws the saved copy number data up to multiplicity 250. Add `-pdf` for PDF output.

Infinite estimated quality means no unsupported assembly kmers were observed. It does not prove that every base, repeat copy or structural join is correct. Read depth holes can hide missing sequence or penalise sequence recovered by another technology. Use the sample's own reads to evaluate its assemblies.

5 Graph construction and exploration

5.1 GGCAT

Build a linked graph from one assembly

```
ggcat build -k 31 -s 1 -e -j 4 -m 1 ASSEMBLY.FA -o GRAPH.FA
```

`GRAPH.FA` stores graph node sequences and links in BCALM style FASTA, ready for Vizitig. `-e` retains links. For assembly inputs, `-s 1` retains sequence that occurs only once.

GGCAT does not remove tips or collapse bubbles. Multiplicity filtering is separate from these graph simplifications.

`-m 1` suggests a 1 GB memory budget for storing temporary data in RAM. This is not a strict limit on total RAM: processing requires additional memory, and GGCAT may exceed the budget. Omitting `-m` uses the default 2 GB. This option is separate from `-s`, the minimum kmer multiplicity.

Build a graph from Illumina reads

```
ggcat build -k 31 -s 3 -e -j 4 -m 1 READS_R1.FQ.gz READS_R2.FQ.gz -o GRAPH.FA
```

`-s 3` filters low count read kmers. This removes many error kmers but can also remove true low depth sequence. GGCAT does not use paired end links here.

Increase the kmer size

```
ggcat build -k 63 -s 3 -e -j 4 -m 1 READS_R1.FQ.gz READS_R2.FQ.gz -o GRAPH.FA
ggcat build -k 127 -s 3 -e -j 4 -m 1 READS_R1.FQ.gz READS_R2.FQ.gz -o GRAPH.FA
```

Choose a different output filename for each graph. Compare branching and repeat resolution, rather than matching node identifiers between graphs.

Build the sequence union of your assembly and the reference

```
ggcat build -k 31 -s 1 -e -j 4 -m 1 ASSEMBLY.FA REFERENCE.FA -o GRAPH.FA
```

Use your assembly FASTA and the reference FASTA as inputs. This identifies shared sequence at the chosen kmer size. Appending native assembly graph files would not do that.

5.2 Bandage (BandageNG)

Open and draw an assembly graph

```
BandageNG load GRAPH.gfa --draw
```

`GRAPH.gfa` is an assembler's graph, for example SPAdes' `assembly_graph_with_scaffolds.gfa` or Flye's `assembly_graph.gfa`. BandageNG also accepts FASTG graphs. Use the graph file, rather than a contig FASTA, to preserve links. Keep the `.gfa` filename extension lowercase for BandageNG. The interactive window requires a desktop display. In the window, zoom into branches, select nodes, inspect their length and depth, and export selected sequences.

Build and open the assembly/reference union graph with GGCAT

```
ggcat build --gfa-v1 -e -k 31 -s 1 -j 4 -m 1 ASSEMBLY.FA REFERENCE.FA -o GRAPH.gfa
BandageNG load GRAPH.gfa --draw
```

`GRAPH.gfa` contains the compacted de Bruijn graph: `--gfa-v1` selects GFA version 1 and `-e` includes graph links. Use one assembly FASTA for a single assembly graph. Change `-k` to compare kmer sizes. This graph represents sequence sharing at the chosen kmer size.
Image export and graph statistics work in Docker without display forwarding.

Export a graph image without opening a window

```
QT_QPA_PLATFORM=offscreen BandageNG image GRAPH.gfa GRAPH.png --width 1600
```

`GRAPH.png` is the image to open first. It contains the drawn graph, not assembly sequences. Use `GRAPH.svg` instead for a vector image.
`QT_QPA_PLATFORM=offscreen` lets Qt render without a desktop display.

Print graph statistics

```
QT_QPA_PLATFORM=offscreen BandageNG info GRAPH.gfa
```

The terminal output includes node and edge counts, connected components, dead ends and node length statistics. Graph node N50 is different from final contig N50.

Colour graph nodes by depth

```
QT_QPA_PLATFORM=offscreen BandageNG image GRAPH.gfa DEPTH.png --colour depth --width 1600
```

`DEPTH.png` colours nodes using the depths stored in the graph. Check that the input graph contains depth information before interpreting the colours.

Show the neighbourhood of a selected node

```
QT_QPA_PLATFORM=offscreen BandageNG image GRAPH.gfa LOCAL.png --scope aroundnodes --nodes NODE_ID --distance 2 --names
```

Replace `NODE_ID` with an exact node identifier from the graph. `LOCAL.png` shows that node and neighbours within two graph steps, with node names. This distance counts links, not bases.

5.3 Vizitig

Set the shell variable `INPUT_FOLDER` to a new input directory name before the following commands. Each subfolder defines an input label: sample and reference. For one assembly, omit the reference subfolder and its copy command. Keep the copied input extensions lowercase.

Build and colour the union directly from your assembly and reference FASTAs

```
export VIZITIG_DIR="$PWD"
mkdir -p "$INPUT_FOLDER/sample" "$INPUT_FOLDER/reference"
cp ASSEMBLY.FA "$INPUT_FOLDER/sample/ASSEMBLY.fasta"
cp REFERENCE.FA "$INPUT_FOLDER/reference/reference.fasta"
vizitig create "$INPUT_FOLDER" -k 31 -f 1 -n GRAPH_NAME --edge-annotation
```

`vizitig create` constructs the graph, merges the inputs, builds the index and adds input colours. No separate `ggcat build` or index command is needed. `-f 1` retains sequence occurring once in an assembly. Choose an unused `GRAPH_NAME`. Keep `VIZITIG_DIR` set to the same writable directory throughout.

Start the Vizitig server

```
vizitig run --hostname 0.0.0.0 --no-launch-browser
```

Open the interface: leave the server running in the terminal. On the same computer, open a web browser and enter http://localhost:4242 in the address bar. This is the address of your local Vizitig server.

Load the graph: select `GRAPH_NAME` in the graph selector. Enter `All` in the query box, then click **Fetch nodes**. The graph and its node table will appear.

The server command works inside the course Docker container. The Docker `-p` option connects port 4242 on your computer to port 4242 inside the container. For a local installation, `vizitig run` can open the browser automatically.

Alternatively, construct and import a GGCAT graph

```
ggcat build -k 31 -s 1 -e -j 4 -m 1 ASSEMBLY.FA REFERENCE.FA -o GRAPH.FA
vizitig build GRAPH.FA -k 31 -n GRAPH_NAME --edge-annotation
vizitig index build GRAPH_NAME
vizitig color GRAPH_NAME -f ASSEMBLY.FA -m sample -c '#0072B2'
vizitig color GRAPH_NAME -f REFERENCE.FA -m reference -c '#D55E00'
```

Choose a fresh `GRAPH_NAME` and run these commands before starting the server. Match `-k` to GGCAT's value. For read inputs, use `READS_R1.FQ.gz` `READS_R2.FQ.gz` and `-s 3` instead of the assembly inputs and `-s 1`.
In the course image, direct `vizitig create` gives an empty graph at k = 127. Use the GGCAT import route above for that size, with `-k 127` in both build commands. Direct construction works at k = 31 and 63.
For abundance information, GGCAT must be built with the `kmer-counters` feature. Vizitig imports the `km:f:` mean kmer abundance annotations when present. The current course image's GGCAT does not output these annotations. The installation guide gives the source build option. Counts from assembly FASTAs describe sequence multiplicity, not sequencing depth.
In the interface, choose **Autocolor → Color → BandHighlight → Apply** to display input membership. Shared nodes may carry both labels.

Find shared sequence: enter this in the query box, then click Fetch nodes

```
Color(sample) AND Color(reference)
```

Find sequence present only in the assembly

```
Color(sample) AND NOT Color(reference)
```

Find sequence present only in the reference

```
Color(reference) AND NOT Color(sample)
```

Inspect a branch: select a node and use **Expand** to fetch its neighbours. **Center**, **Linearize** and **Orient** help arrange the local graph. Inspect node sequences and lengths in the table.
Compare a single nucleotide variant: find two short bubble arms between shared flanks. Select their nodes in the table and use **Open aligner** to compare the sequences.
Investigate structural differences: inspect shared flanks, alternative paths, junctions and orientation. A sample specific path may also reflect missing coverage or an assembly error. Colours alone do not establish repeat copy number.
Keep the evidence: name and **Save** a useful graph view. Keep the node identifiers, sequences and input source for your interpretation.

6 Practical speedrun

6.1 Assemble

If using Docker, start the container with the command on the first page before following this speedrun. This is one possible route through the practical. Use the supplied sample reads. Choose different output filenames and prefixes for separate runs, and unused Vizitig graph names. Uppercase names are placeholders.

Check the reads

```
seqkit stats -a READS_R1.FQ.gz READS_R2.FQ.gz PACBIO.FQ.gz NANOPORE.FQ.gz
seqkit seq REFERENCE.FA.gz > REFERENCE.FA
```

Compare pairing and kmer size with SPAdes

```
spades.py --only-assembler -1 READS_R1.FQ.gz -2 READS_R2.FQ.gz -k 31 -t 4 -m 8 -o PAIRED_31
spades.py --only-assembler --s1 READS_R1.FQ.gz --s2 READS_R2.FQ.gz -k 31 -t 4 -m 8 -o UNPAIRED_31
spades.py --only-assembler -1 READS_R1.FQ.gz -2 READS_R2.FQ.gz -k 127 -t 4 -m 8 -o PAIRED_127
spades.py --only-assembler --s1 READS_R1.FQ.gz --s2 READS_R2.FQ.gz -k 127 -t 4 -m 8 -o UNPAIRED_127
```

Compare `contigs.fasta` first, then `scaffolds.fasta`, from each output. Check whether pairing and larger kmers improve contiguity.

Compare short read assemblers

```
megahit -1 READS_R1.FQ.gz -2 READS_R2.FQ.gz -t 4 -m 8589934592 -o MEGAHIT_OUTPUT
```

Use `final.contigs.fa` for the comparison.

Assemble long reads with Flye and hifiasm

```
flye --nano-hq NANOPORE.FQ.gz --genome-size 500k --threads 4 --out-dir FLYE_NANOPORE
flye --pacbio-hifi PACBIO.FQ.gz --genome-size 500k --threads 4 --out-dir FLYE_PACBIO
hifiasm -o HIFIASM -t 4 -l0 -f0 PACBIO.FQ.gz
awk '$1=="S" && $3!="*" {print ">"$2; print $3}' HIFIASM.bp.p_ctg.gfa > HIFIASM.FA
```

Use Flye's `assembly.fasta` and `assembly_info.txt`, then `HIFIASM.FA`. Compare contiguity and inspect circularity in Flye's table.

6.2 Evaluate

Compare assembly lengths and N50

```
seqkit stats -a ASSEMBLY_1.FA ASSEMBLY_2.FA ASSEMBLY_3.FA
```

Repeat for the remaining assemblies. Compare contigs of at least 500 bases consistently. Use `seqkit seq -m 500 ASSEMBLY.FA | seqkit stats -a` for a filtered check.

Compare assemblies with the related reference

```
quast.py ASSEMBLY_1.FA ASSEMBLY_2.FA ASSEMBLY_3.FA -r REFERENCE.FA --min-contig 500 --threads 4 -o QUAST_OUTPUT
```

Open `report.html` first, then `icarus.html`. Compare genome fraction, duplication, misassemblies and NGA50. The reference differs from the sample: disagreements are not automatically assembly errors.

Build read evidence for kmer based evaluation

```
FastK -k21 -t1 -M2 -T4 -NREADS READS_R1.FQ.gz READS_R2.FQ.gz
Histex -A -h1:500 READS > READS.hist.tsv
```

Use this database from the sample's Illumina reads to evaluate its assemblies.

Run MerquryFK on each assembly

```
MerquryFK -T4 READS ASSEMBLY.FA EVALUATION
```

Use a new output prefix for each assembly, with the FastK table, histogram and hidden table files kept together. Inspect `EVALUATION.qv`, `EVALUATION.completeness.stats` and the copy number spectra plots. Record quality and completeness alongside N50.

6.3 Optional hybrid assemblies

Combine PacBio and Nanopore reads

```
hifiasm -o HYBRID -t 4 -l0 -f0 --ul NANOPORE.FQ.gz PACBIO.FQ.gz
awk '$1=="S" && $3!="*" {print ">"$2; print $3}' HYBRID.bp.p_ctg.gfa > HYBRID.FA
verkko -d VERKKO_OUTPUT --hifi PACBIO.FQ.gz --nano NANOPORE.FQ.gz --no-correction --no-consensus-bam --haploid --local-cpus
4 --local-memory 8
```

Evaluate `HYBRID.FA` and Verkko's `assembly.fasta` in the same way. Compare with the assemblies from each technology alone.

6.4 Explore graphs and compare genomes

Build Illumina graphs at three kmer sizes

```
ggcat build -k 31 -s 3 -e -j 4 -m 1 READS_R1.FQ.gz READS_R2.FQ.gz -o READ_GRAPH_31.FA
ggcat build -k 63 -s 3 -e -j 4 -m 1 READS_R1.FQ.gz READS_R2.FQ.gz -o READ_GRAPH_63.FA
ggcat build -k 127 -s 3 -e -j 4 -m 1 READS_R1.FQ.gz READS_R2.FQ.gz -o READ_GRAPH_127.FA
```

Inspect an assembler's native graph with Bandage

```
QT_QPA_PLATFORM=offscreen BandageNG info GRAPH.gfa
QT_QPA_PLATFORM=offscreen BandageNG image GRAPH.gfa GRAPH.png --width 1600
```

Use the GFA graph from SPAdes or Flye. Open `GRAPH.png` to inspect the complete graph.

Set the shell variables ASSEMBLY_INPUT and UNION_INPUT to different new input directory names before the following commands.

Import the read graphs and build an assembly graph directly from FASTA

```
export VIZITIG_DIR="$PWD"
vizitig build READ_GRAPH_31.FA -k 31 -n READ_31 --edge-annotation
vizitig index build READ_31
vizitig build READ_GRAPH_63.FA -k 63 -n READ_63 --edge-annotation
vizitig index build READ_63
vizitig build READ_GRAPH_127.FA -k 127 -n READ_127 --edge-annotation
vizitig index build READ_127
mkdir -p "$ASSEMBLY_INPUT/sample"
cp ASSEMBLY.FA "$ASSEMBLY_INPUT/sample/ASSEMBLY.fasta"
vizitig create "$ASSEMBLY_INPUT" -k 31 -f 1 -n ASSEMBLY_GRAPH --edge-annotation
```

Build, index and colour the assembly/reference union

```
mkdir -p "$UNION_INPUT/sample" "$UNION_INPUT/reference"
cp ASSEMBLY.FA "$UNION_INPUT/sample/ASSEMBLY.fasta"
cp REFERENCE.FA "$UNION_INPUT/reference/reference.fasta"
vizitig create "$UNION_INPUT" -k 31 -f 1 -n UNION --edge-annotation
vizitig run --hostname 0.0.0.0 --no-launch-browser
```

ASSEMBLY.FA is an assembly you produced. REFERENCE.FA is the supplied related reference. Vizitig graph names must be unused in your data store.

In the browser: open http://localhost:4242 on the computer running the server. Docker requires port 4242 to be published. Leave the terminal running.

- Select each read graph. Query `All` and click **Fetch nodes**. Compare branches and repeat resolution across kmer sizes.
- Select ASSEMBLY_GRAPH and inspect unresolved branches. Select UNION. Fetch `All`. Choose **Autocolor → Color → BandHighlight → Apply**.
- Query `Color(sample) AND NOT Color(reference)` or `Color(reference) AND NOT Color(sample)`. Expand neighbours to find shared flanks.
- Compare bubble arm sequences with **Open aligner**. Investigate a candidate single nucleotide variant and a structural difference.
- Save a graph view and record node identifiers, assembly sources and evidence. Consider missing coverage and assembly errors before calling a variant.